

JavaScript Notes — Day 8

Chapter 8: Events

What is an Event?

A change in the state of an object is known as an event. Events are fired to notify code of "interesting changes" that may affect code execution — things like a click, a key press, or a form submission.

Common Event Types

- Mouse events — click, dblclick, mouseover, mouseout, etc.
- Keyboard events — keydown, keyup, keypress
- Form events — submit (and others)
- Print events — beforeprint, afterprint

Attaching an Event in HTML

The quickest way — write the handler directly as an attribute on the tag:

```
<button onclick="console.log('button was clicked');">
  click me
</button>
```

Note: this inline style works, but mixing JS logic into HTML attributes gets messy fast — the JS-side approach below (event handling / listeners) is the preferred, scalable way.

Event Handling in JS

Instead of writing the handler in HTML, select the element and assign a function to its event property directly:

```
node.event = (e) => {
  // handle here
};
```

Example:

```
let button = document.querySelector("button");

button.onclick = (e) => {
  console.log("button was clicked");
};
```

Note: assigning like `button.onclick = fn1` and later `button.onclick = fn2` overwrites the first handler — an element can only have one "onclick"-style handler at a time this way. `addEventListener` (below) fixes that.

The Event Object

It is a special object that holds details about the event. All event handlers automatically get access to this object's properties and methods — usually passed in as the parameter (commonly named `e` or `evt`).

Property	Gives you
<code>e.target</code>	The exact element the event happened on
<code>e.type</code>	The type of event (e.g. "click")
<code>e.clientX</code>	Mouse X position (relative to the browser window)
<code>e.clientY</code>	Mouse Y position (relative to the browser window)

Event Listeners

`addEventListener` lets you attach multiple handlers to the same event on the same element — unlike `node.onclick = fn`, none of them overwrite each other.

```
node.addEventListener(event, callback);
node.removeEventListener(event, callback);
```

Note: to remove a listener, the callback passed to `removeEventListener` must be the exact same function reference that was passed to `addEventListener` — an anonymous arrow function written inline can never be removed later, since there's no saved reference to it.

Example

```
btn.addEventListener("click", (evt) => {
  console.log("button was clicked", evt);
  console.log(evt.type);
});
```

`node.onclick` vs. `addEventListener`

	<code>node.onclick = fn</code>	<code>node.addEventListener("click", fn)</code>
Multiple handlers?	No — last one wins	Yes — all run
Can be removed individually?	Just overwrite with null	Yes, with <code>removeEventListener</code>
Common use	Quick, one-off handler	Preferred for real apps

Practice Problem — Dark Mode Toggle Button

Create a toggle button that switches the page between dark and light mode. Toggle means flipping between two states each click (state 1 → state 2 → state 1 ...).

Approach 1 — directly changing style (manual, state needs tracking):

```
let modeBtn = document.querySelector(".mode-btn");

document.querySelector("body").style.backgroundColor = "...";
// would need an if/else + a tracked boolean to flip each click
```

Approach 2 — using classList (cleaner, CSS handles the styling):

```
body.classList.add("dark");
```

Full solution — `classList.toggle()` automatically adds the class if it's missing and removes it if it's already there, which is exactly what a toggle button needs:

```
let modeBtn = document.querySelector(".mode-btn");
let body = document.querySelector("body");

modeBtn.addEventListener("click", () => {
  body.classList.toggle("dark");
});

/* in CSS */
body.dark {
  background-color: #191919;
  color: #ffffff;
}
```

This way no manual state variable or if/else is needed — the presence/absence of the "dark" class on body is the state, and `classList.toggle()` flips it on every click.

Quick Reference

Task	Code
Inline HTML handler	<code>onclick="..."</code> attribute on the tag
Assign handler from JS	<code>node.onclick = (e) => { ... }</code>
Attach a listener	<code>node.addEventListener(event, callback)</code>
Remove a listener	<code>node.removeEventListener(event, callback)</code>
Element that triggered the event	<code>e.target</code>
Type of event	<code>e.type</code>
Add / remove a class	<code>el.classList.add()</code> / <code>el.classList.remove()</code>
Flip a class on/off	<code>el.classList.toggle("className")</code>